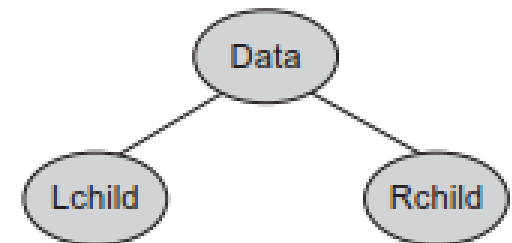
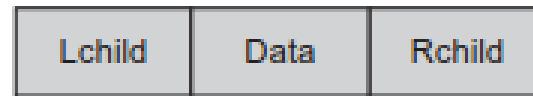


Binary Tree Representation

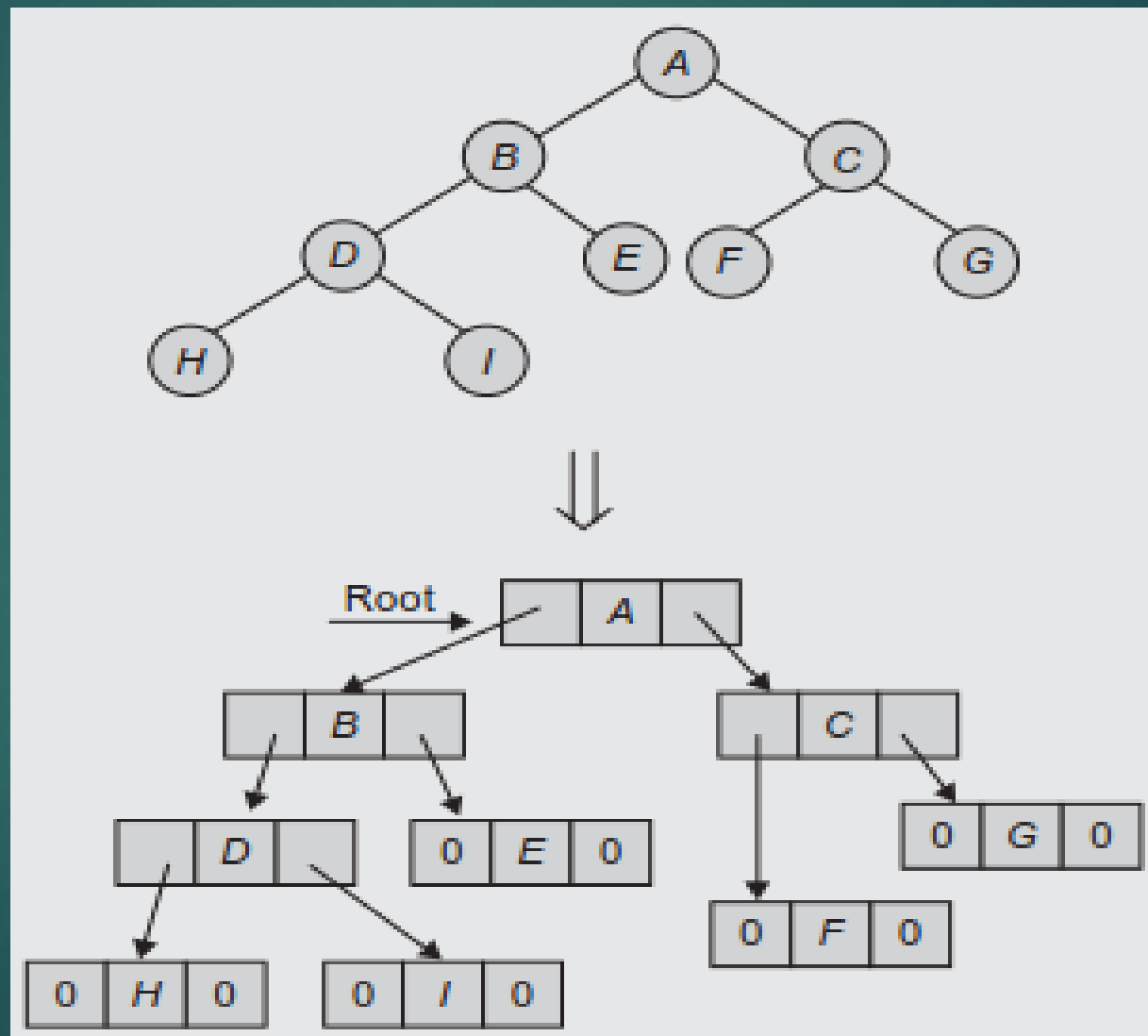
- ▶ Linked list
- ▶ Array

Linked Implementation of Binary Tree

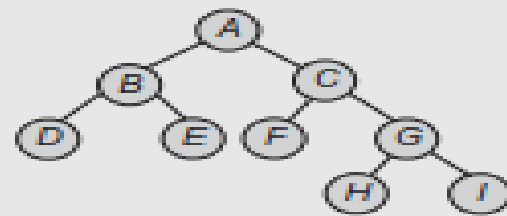
- ▶ we need each node with data and link fields.
- ▶ Each node of a binary tree has both a left and a right subtree.
- ▶ Each node will have three fields—Lchild, Data, and Rchild.
- ▶ In this node structure, Lchild and Rchild are the two link fields to store the addresses of left child and right child of a node; data is the information content of the node.
- ▶ With this representation, if we know the address of the root node, then using it, any other node can be accessed.



Linked representation



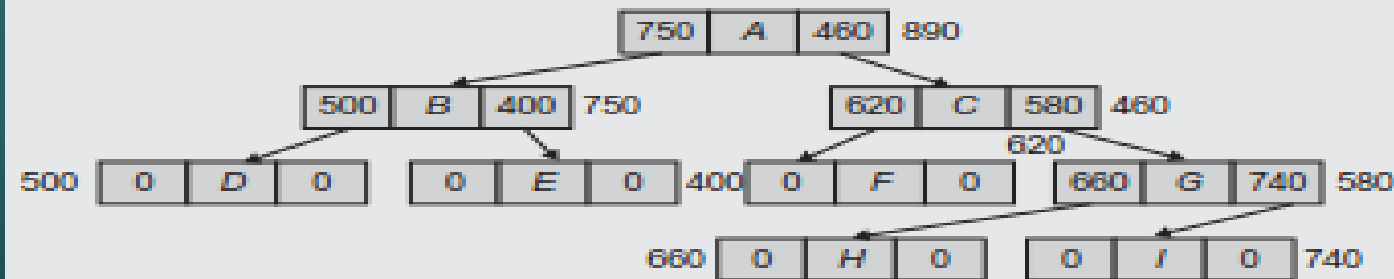
Linked representation



(a)

Address	Nodes		
	Child	Data	Rchild
500	0	D	0
750	500	B	400
400	0	E	0
890	750	A	460
620	0	F	0
460	620	C	580
660	0	H	0
580	660	G	740
740	0	I	0

(b)



(c)

Linked Representation

► Class TreeNode

```
{  
    public:  
    char Data;  
    TreeNode *Lchild;  
    TreeNode *Rchild;  
};
```

Class BinaryTree{

private:

TreeNode *Root;

public:

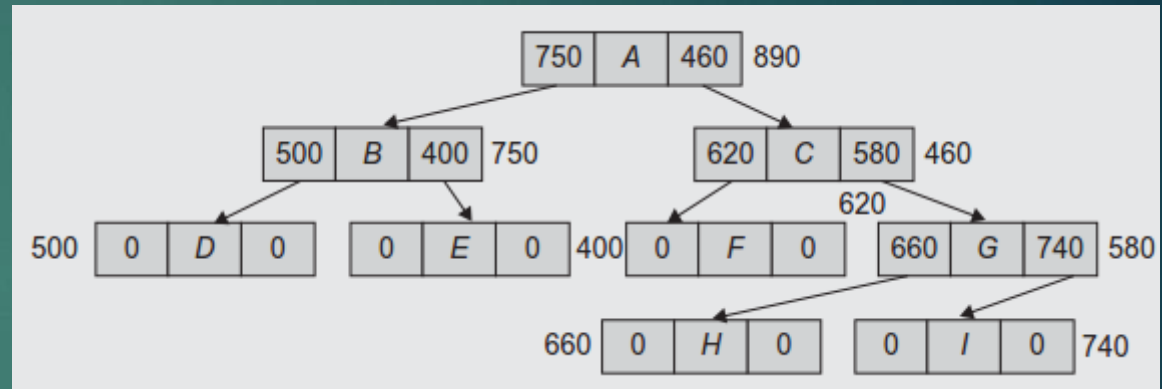
BinaryTree() {Root = Null;}

TreeNode *Create()

}

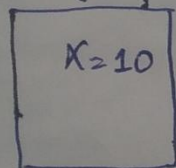
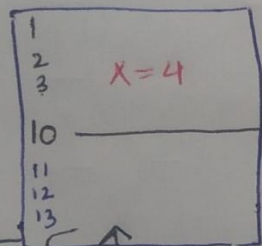
Binary Tree Creation

```
TreeNode *Create()
{
    Int x;
    TreeNode *NewNode;
    newnode = new TreeNode;
    Cout<<"enter the data,-1 for no node"
    Cin>>x;
    If(x == -1)
        Return 0;
    Newnode->data=x;
    Cout<<"enter left child of x"
    Newnode->left=create();
    Cout<<"enter the right child of x";
    Newnode->right=create();
    Return newnode;
}
```

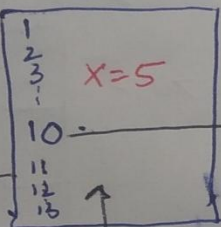


root

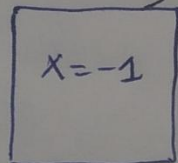
new node 100



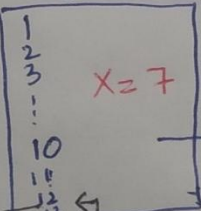
new node = 200



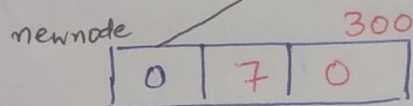
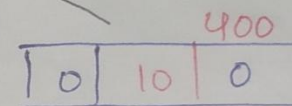
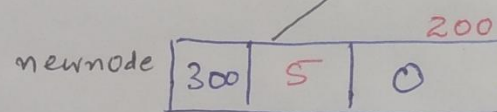
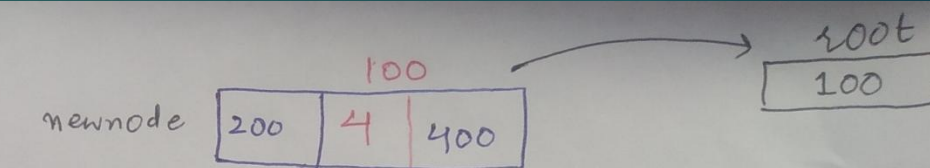
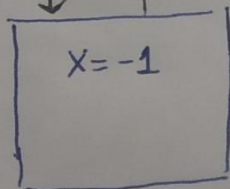
return 200



new node = 300



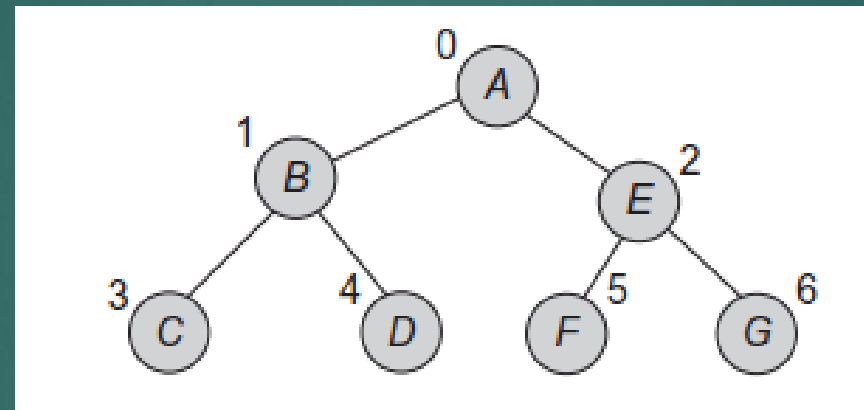
return 300



Array Implementation of Binary Tree

- ▶ One of the ways to represent a tree using an array is to store the nodes level-by-level, starting from the level 0 where the root is present. Such a representation requires sequential numbering of the nodes, starting with the nodes on level 0, then those on level 1, and so on.
- ▶ The root node is stored in the first memory location as the first element in the array.
- ▶ 1. $\text{Parent}(i) = \lfloor (i-1)/2 \rfloor$ if $i \neq 0$, if $i=0$, then it is the root that has no parent.
- ▶ 2. $\text{Lchild}(i) = 2 \times i + 1$ if $2i + 1 \leq n - 1$; if $2i \geq n$, then i has no left child.
- ▶ 3. $\text{Rchild}(i) = 2i + 2$ if $2i + 2 \leq n - 1$; if $(2i + 1) \geq n$, then i has no right child.

Array Implementation of Binary Tree

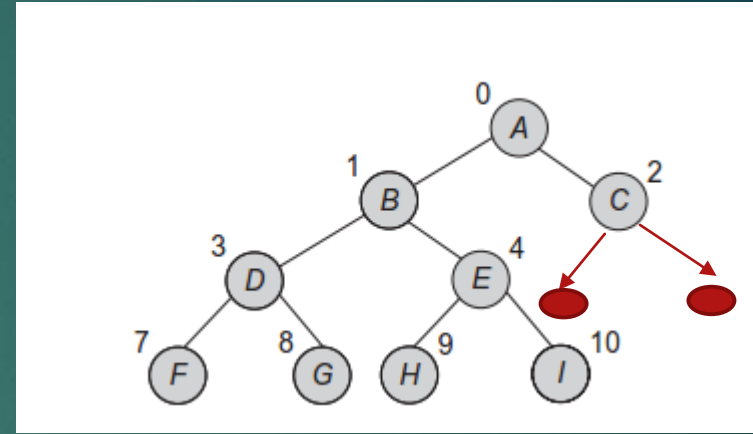
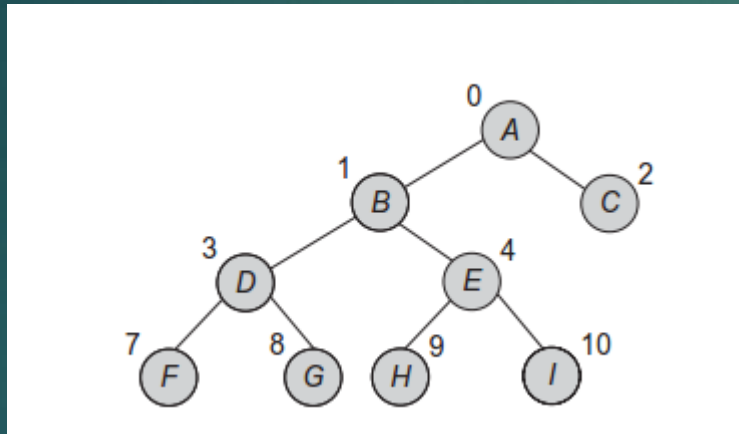


0	1	2	3	4	5	6
A	B	E	C	D	F	G

Array Implementation of Binary Tree

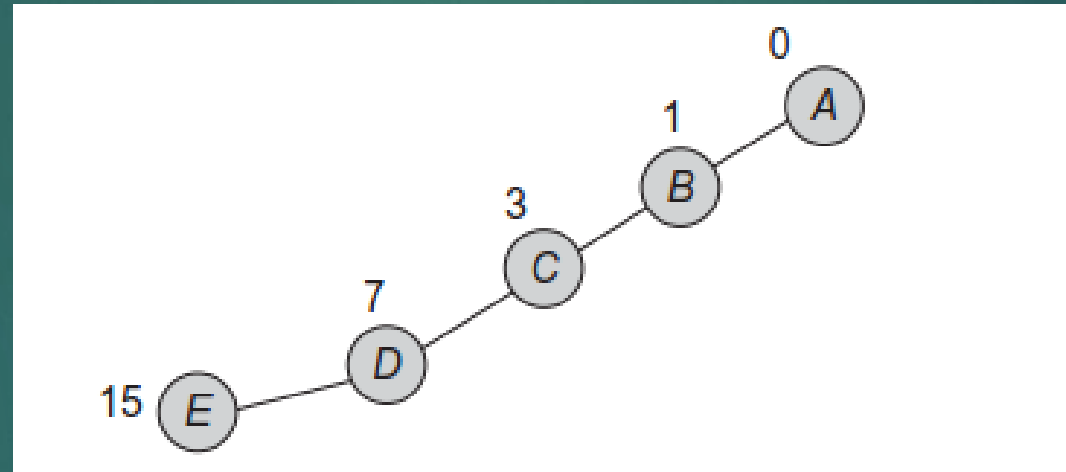
- ▶ 1. $\text{Parent}(i) = \lfloor i/2 \rfloor$
- ▶ 2. $\text{Lchild}(i) = 2 \times i$
- ▶ 3. $\text{Rchild}(i) = 2i + 1$

Array Implementation



A	B	C	D	E	-	-	F	G	H	I
0	1	2	3	4	5	6	7	8	9	10

Array Implementation



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	B	-	C	-	-	-	D	-	-	-	-	-	-	-	E

Binary Tree Traversal

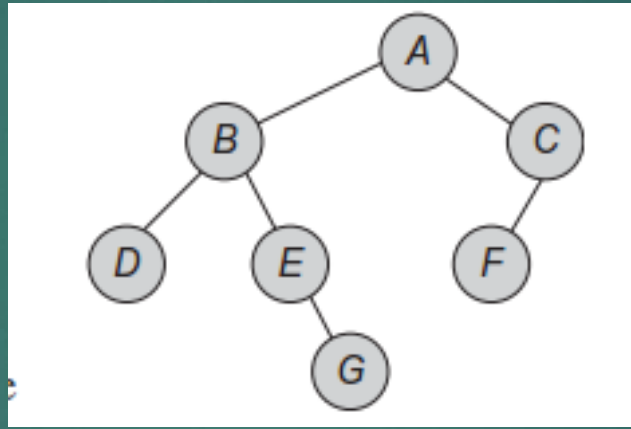
- ▶ Inorder traversal
- ▶ Postorder traversal
- ▶ Preorder traversal

Preorder Traversal

Preorder Traversal

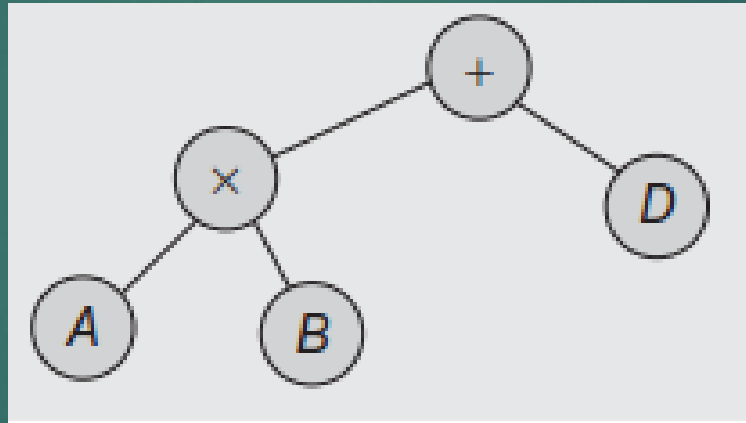
- ▶ In this traversal, the root is visited first followed by the left subtree in preorder and then the right subtree in preorder.
- ▶ **Preorder (DLR) Algorithm**
 1. Visit the root node, say D.
 2. Traverse the left subtree of the node in preorder.
 3. Traverse the right subtree of the node in preorder.

Preorder Traversal

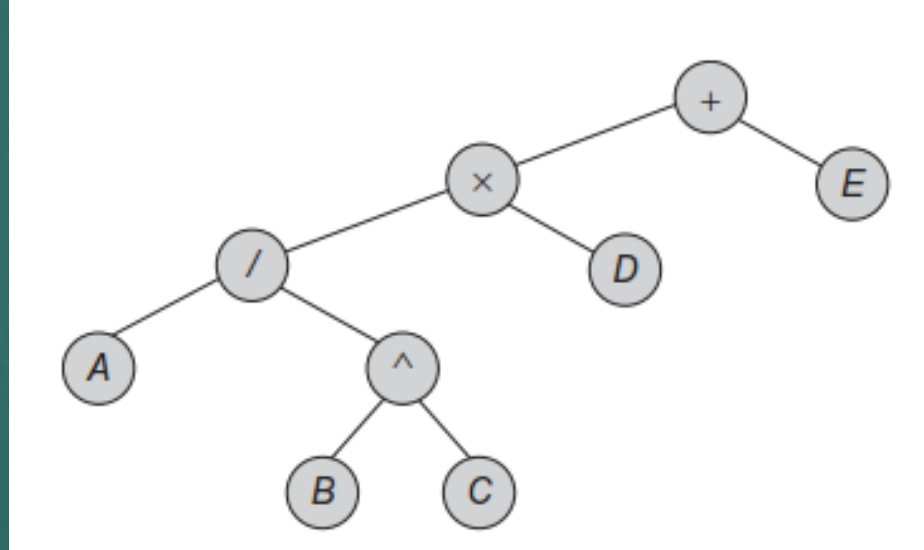


ABDEGCF.

Preorder Traversal



Preorder Traversal

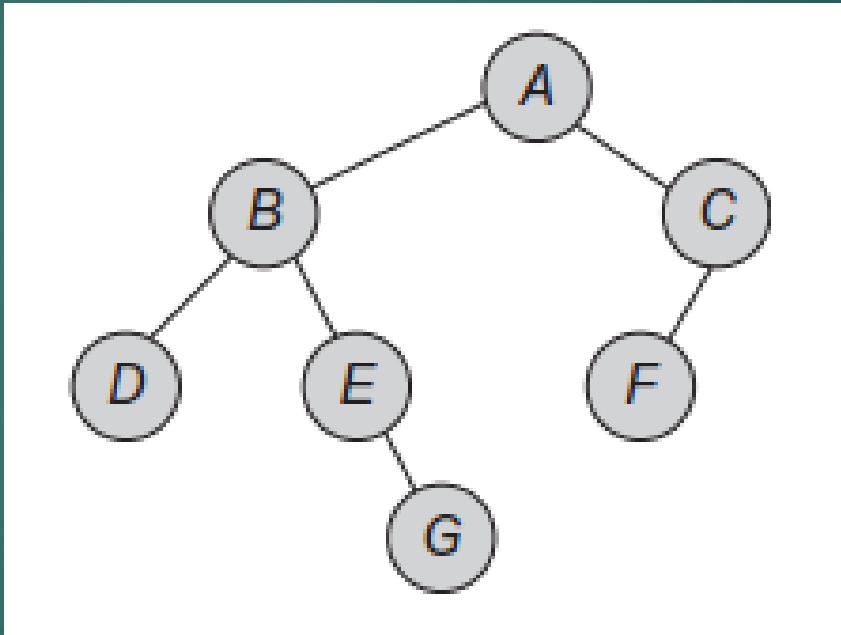


Inorder Traversal

inorder Traversal

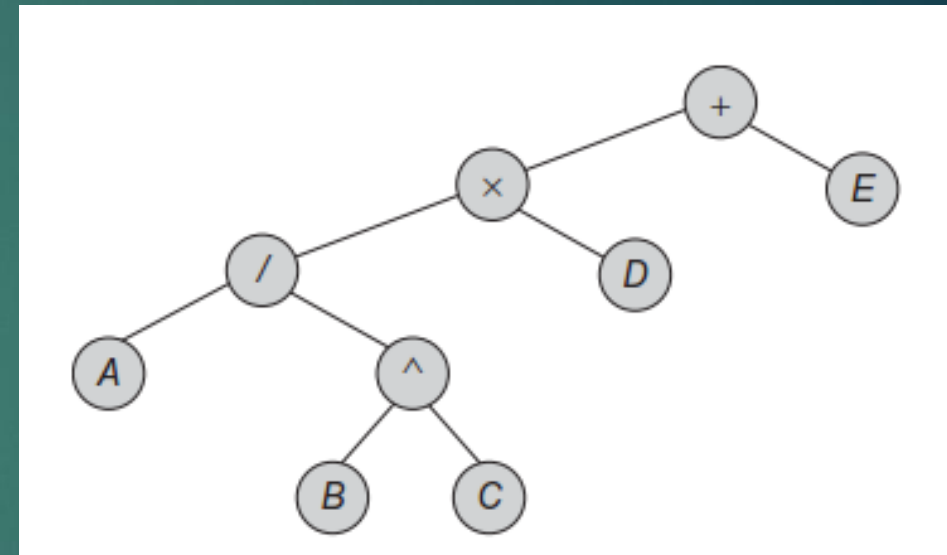
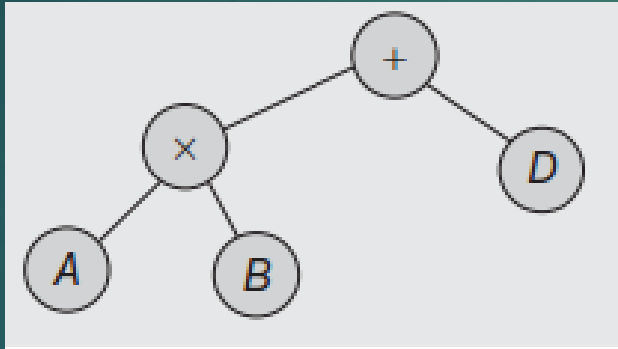
- ▶ In this traversal, the left subtree is visited first in inorder followed by the root and then the right subtree in inorder.
- ▶ **Inorder (LDR) Algorithm**
 1. Traverse the left subtree of the root node in inorder.
 2. Visit the root node node.
 3. Traverse the right subtree of the root node in inorder.

Inorder Traversal



DBEG AFC

Inorder Traversal



Postorder Traversal

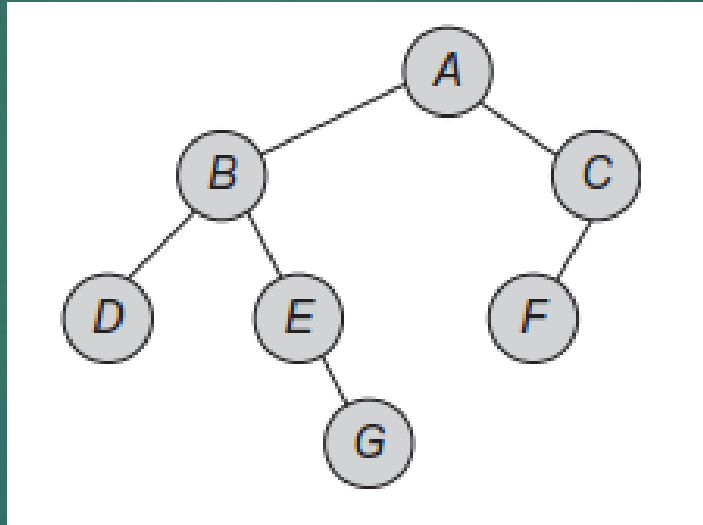
► postorder Traversal

In this traversal, the left subtree is visited first in postorder followed by the right subtree in postorder and then the root. This is defined as the following:

► Postorder (LRD) Algorithm

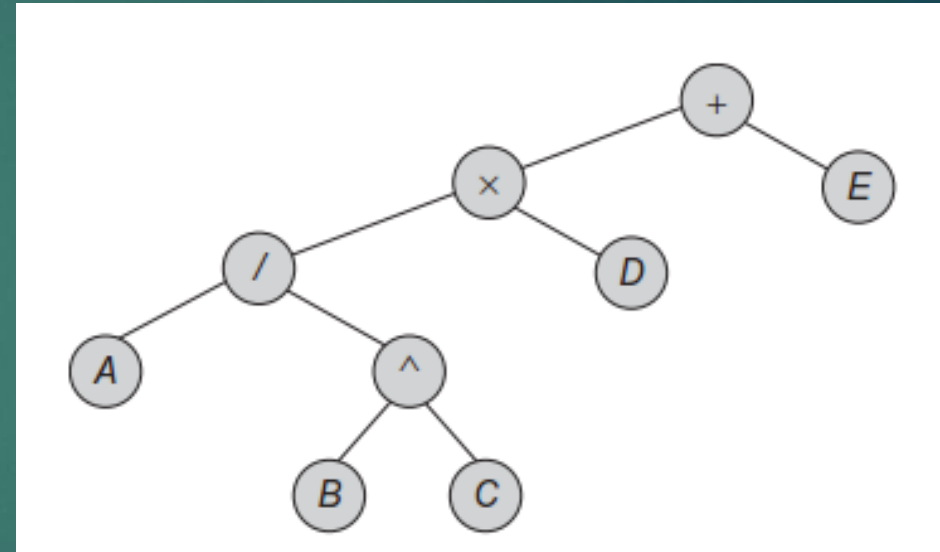
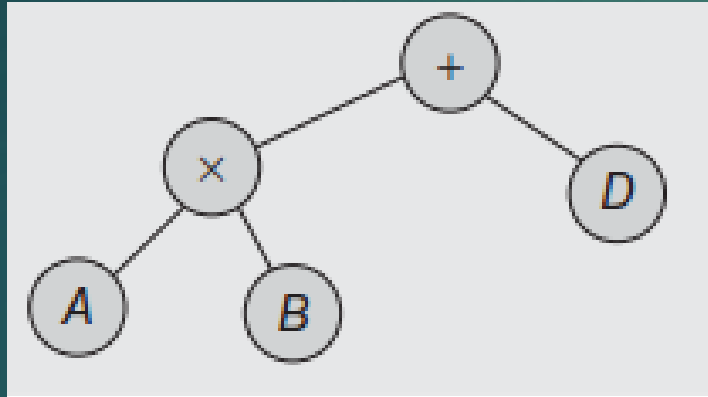
1. Traverse the root's left child (subtree) of the root node in postorder.
2. Traverse the root's right child (subtree) of the root node in postorder.
3. Visit the root node.

Postorder Traversal



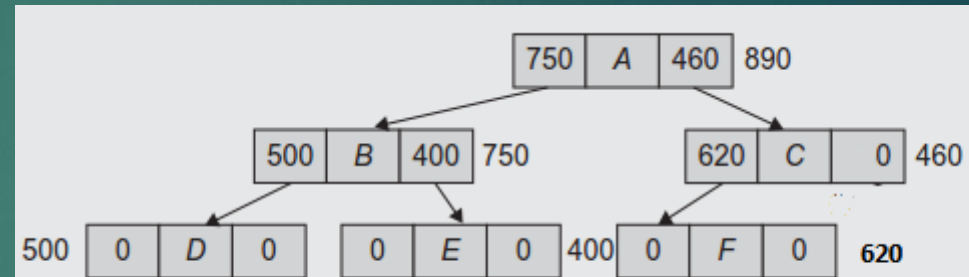
D G E B F C A

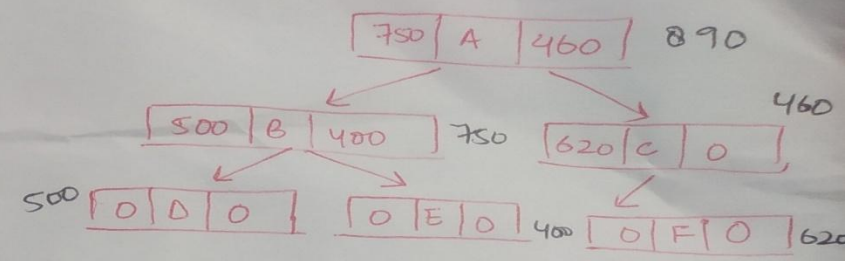
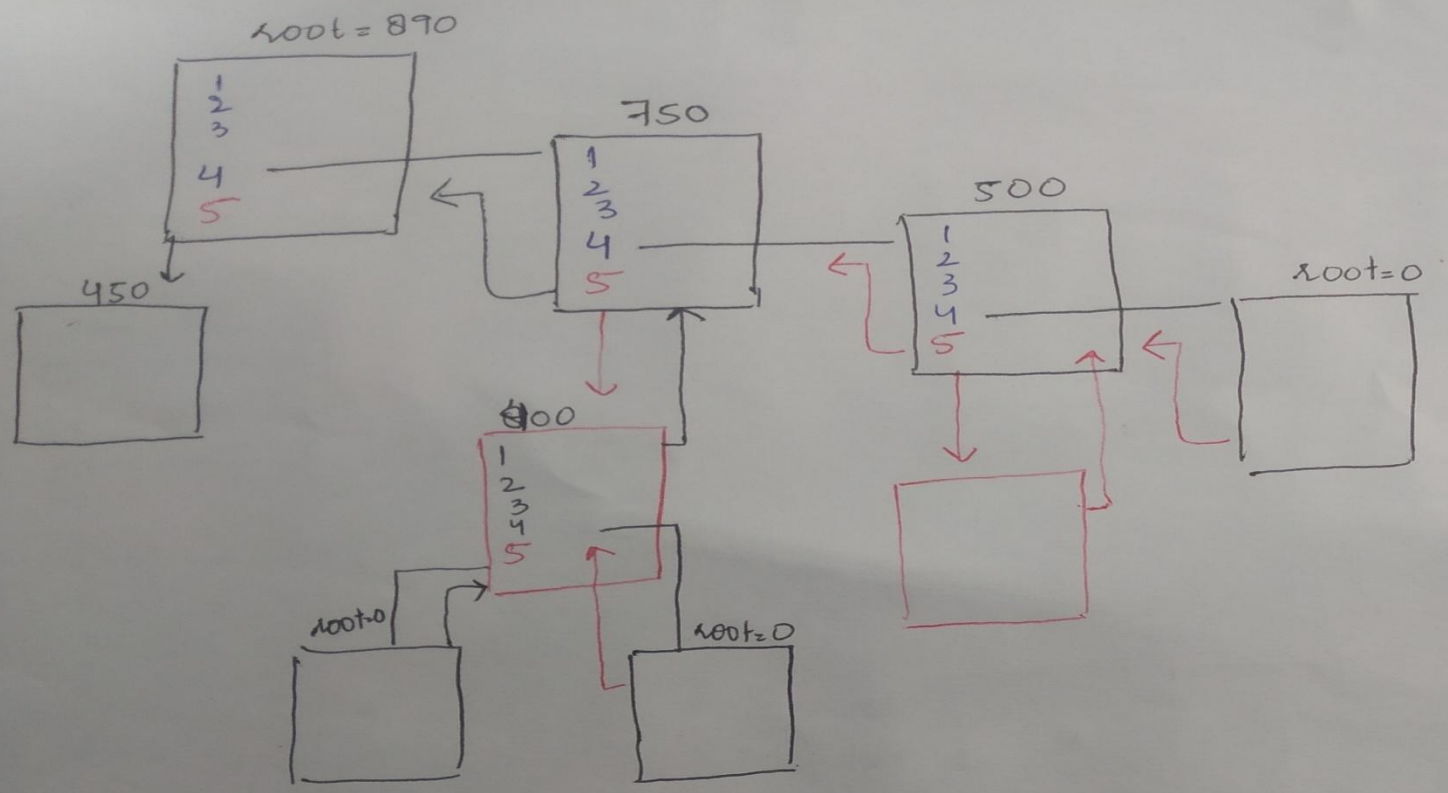
Postorder Traversal



Preorder

```
void BinaryTree :: Preorder(TreeNode*)  
{  
    if(Root == Null)  
        return;  
    cout << Root->Data;  
    Preorder(Root->Lchild);  
    Preorder(Root->Rchild);  
}
```





A B D E C F

Inorder

```
void BinaryTree :: Inorder(TreeNode*)  
{  
    if(Root == Null)  
        return;  
    Inorder(Root->Lchild);  
    cout << Root->Data;  
    Inorder(Root->Rchild);  
}
```

Postorder

```
void BinaryTree :: Postorder(TreeNode*)  
{  
    if(Root == Null)  
        return;  
    Postorder(Root->Lchild);  
    Postorder(Root->Rchild);  
    cout << Root->Data;  
}
```